

— Objetivos —

- Entender o uso dos operadores bitwise na manipulação de bits.
- Utilizar operadores bitwise em tarefas de otimização e controle de bits.

— Exercícios —

- 1) Utilizando operadores bitwise, faça uma função que verifique se um número é ímpar.
- 2) Utilizando operadores bitwise, faça uma função que troque duas variáveis.
- 3) Utilizando operadores bitwise, faça uma função que converta uma letra para maiúscula.
- 4) Utilizando operadores bitwise, faça uma função que converta uma letra para minúscula.
- 5) Faça uma função que escreva os bits de um número inteiro de 32 bits.
- 6) O operador de deslocamento à esquerda pode ser usado para compactar dois valores de caracteres em uma variável unsigned short int (2 bytes). Escreva um programa que leia 2 caracteres do teclado e passe para a função compactaCaracteres. Para compactar dois caracteres em uma variável inteira unsigned short int, atribua o primeiro caractere a variável unsigned short int, desloque a variável para esquerda em 8 posições de bits e combine a variável unsigned com o segundo caractere usando o operador OR sobre bits.
- 7) Usando o operador deslocamento à direita e o operador AND sobre bits e uma máscara, escreva uma função descompactaCaracteres que recebe o inteiro unsigned short int e o descompacte em dois caracteres.

8) Faça uma função

```
int crossover(int n, int m)
```

que retorna um inteiro que representa os 16 bits mais significativos de n e os 16 bits menos significativos de m. Por exemplo:

```
int main() {
    int x;

    x = crossover(305419896, 1737075661);
    // 30541989610 = 1234567816 = 0001001000110100 0101011001111000
    // 173707566110 = 6789ABCD16 = 0110011110001001 1010101111001101

    // x receberá o valor
    // 30544174110 = 1234ABCD16 = 0001001000110100 1010101111001101
}
```

Faça também um programa que leia 2 inteiros, chame a função com os respectivos valores e receba o seu retorno. O programa deve mostrar os números em decimal e binário, como segue:

```
305419896  0x12345678
1737075661 0x6789ABCD
305441741  0x1234ABCD
```

9) Escreva uma função que receba 3 parâmetros (hora, minuto e segundo) e devolva pelo nome os 3 valores codificados nos bytes menos significativos: o byte 3 deve ter o valor 0, o byte 2 deve armazenar a hora, o byte 1 deve armazenar o minuto e o byte 0 deve armazenar os segundos (o byte 3 é o mais significativo e o byte 0 é o menos significativo). O protótipo da função é dado por:

```
int compactaHora(int hora,
                 int minuto,
                 int segundo);
```

Faça um programa que leia a hora, minuto e segundo e mostre a hora compactada em hexadecimal. Para o caso da seguinte chamada

```
int horacodificada = compactaHora(14, 32, 25);
```

a variável horacodificada deve ter o seguinte conteúdo:

```
00000000 00001110 00100000 00011001
```

pois 14 em binário é 1110_2 , 32 em binário é 10000_2 e 25 em binário é 11001_2 . O programa deve imprimir na tela o valor compactada em hexadecimal e em decimal:

```
0x000E2019    925721
```

10) A função

```
void descompactaHora(int horaCompactada,
                    int *hora,
                    int *minuto,
                    int *segundo);
```

recebe uma hora compactada como parâmetro e devolve em hora, minuto e segundo as respectivas informações extraídas da hora compactada. A hora compactada armazena a hora, minuto e segundo codificados nos bytes 2, 1 e 0 (onde o byte 3 é o mais significativo e o byte 0 é o menos significativo). Por exemplo, a hora compactada, cujo valor em decimal é 925721, armazena a hora 14:32:25. Para melhor entender esta codificação de valores em um único valor, é necessário converter o valor 925721 em binário utilizando 32 bits:

```
Byte 3    Byte 2    Byte 1    Byte 0
00000000 00001110 00100000 00011001
```

Note que o Byte 2 possui o valor binário 00001110_2 que convertido em decimal representa 14 (hora). O Byte 1 possui o valor binário 00100000_2 que convertido em decimal representa 32 (minuto). Finalmente, o Byte 0 possui o valor binário 00011001_2 que convertido em decimal representa 25 (segundos).

Implemente esta função e escreva um programa que leia um inteiro sem sinal e mostre a hora codificada nele. O programa deve mostrar todos os números (hora, minuto e segundo) em decimal.

11) Faça a função

```
char rodaParaEsquerda(char n, char nBits)
```

que retorna o n com nBits rotações à esquerda. A rotação garante que os bits à esquerda serão copiados para os bits menos significativos de n, garantindo que nenhum bit é perdido (como ocorre no deslocamento). Por exemplo, para a seguinte chamada

```
char m = rodaParaEsquerda(77, 3);
```

a variável m receberá o valor 108 (em binário 1101100) pois como o valor 77 em binário é 1001101, os 3 bits mais significativos após a rotação transformaram-se nos 3 menos significativos.

Faça também um programa que leia um número e o número de bits a serem deslocados. O programa deve mostrar o número original e o número rotacionado em decimal.

12) O desenho da fonte de caracteres 8x8 dos valores ASCII pode ser representado por 8 bytes. Cada byte corresponde a uma das linhas do caractere e os bits de cada byte definem se aquela posição está ligada (valor 1) ou desligada (valor 0). Por exemplo, os 8 bytes a seguir definem a representação da letra Y neste formato:

```
0 1 1 1 0 0 1 1 1
1 1 1 1 0 0 1 1 1
2 1 1 1 0 0 1 1 1
3 0 1 1 1 1 1 1 0
4 0 0 1 1 1 1 0 0
5 0 0 1 1 1 1 0 0
6 0 1 1 1 1 1 1 0
7 1 1 1 1 1 1 1 1
```

Escreva uma função que receba como argumento um vetor de 8 unsigned char e escreva na tela o caractere representado no vetor. Quando o bit estiver ligado, deve-se imprimir o caractere '#' e quando desligado, deve-se imprimir o caractere espaço ' '. O protótipo da função é:

```
void imprimeASCII(unsigned char caractere[8]);
```

Para a representada do caractere Y dado pelo exemplo anterior, a função deverá escrever na tela o seguinte:

```
###  ###
###  ###
###  ###
#####
####
####
#####
```

Faça também um programa que crie o vetor de char e chame a função implementada.

Solução dos Exercícios sobre Operadores Bitwise

Operadores Bitwise - Trabalho Discente Efetivo - TDE

1) Uma técnica muito utilizada para a identificação de erros é o teste de paridade, o qual pode ser descrito da seguinte forma: Seja uma mensagem que será enviada de um dispositivo para outro. Antes da mensagem ser enviada, um bit extra é adicionado no final da representação binária. Se a mensagem tiver um número par de bits de valor 1, o bit extra terá valor 0. Caso contrário, se a mensagem tiver um número ímpar de bits de valor 1, bit extra terá o valor 1. Quando o destinatário recebe a mensagem ele faz a contagem de bits de valor 1. Se a quantidade for par, significa que a mensagem chegou com sucesso. Caso contrário, significa que a mensagem sofreu uma alteração e não está correta.

Faça uma função que receber um unsigned char (8 bits), em que os 7 primeiros bits correspondem as informações a serem transmitidas. A função deverá calcular e inserir, o bit de paridade (bit menos significativo), de forma que após a inserção a mensagem tenha um número par de bits de valor 1. A função deverá retornar o valor alterado. O protótipo da função é:

```
unsigned char calcula_bit_de_paridade(unsigned char valor)
```

Exemplos de Entrada	Exemplos de Saída
11000100	11000101
11011000	11011000
10000010	10000010
10000110	10000111

2) Considere o armazenamento de um horário de forma compactada em uma única variável do tipo unsigned int (32 bits) onde: os primeiros 14 bits permanecem zerados; os 6 bits seguintes são usados para armazenar a hora, os próximos 6 bits são utilizados para armazenar os minutos; e os últimos 6 bits são usados para armazenar os segundos. Por exemplo, considerando que tenhamos o seguinte horário:

```
Hora: 13
Minuto: 45
```

Segundo: 30

A representação binária em uma variável do tipo unsigned int (32 bits) seria:

	Hora	Minuto	Segundo
0000000000000000	001101	101101	011110

Faça uma função que receba um horário neste formato e retorne um horário no mesmo formato, mas com o valor correspondente ao segundo seguinte. Por exemplo, para o exemplo acima a função deverá retornar:

	Hora	Minuto	Segundo
0000000000000000	001101	101101	011111

OBS: para o horário 23:59:59, o horário correspondente ao segundo seguinte seria 00:00:00.